Cuprins

1 Noțiuni generale **3**

1.1 Alfabet, Șir, Limbaj . . . 3
1.1.1 Alfabet . . . 3
1.1.2 Șir . . . 3
1.1.3 Limbaj . . . 3
1.2 Operații pe limbaje . . . 4
1.2.1 Reuniune . . . 4
1.2.2 Concatenare . . . 4
1.2.3 Kleene Star . . . 4
1.2.4 Mulțimea putere . . . 5
1.3 Ierarhia Chomsky . . . 5

2 Limbaje regulate **6**

2.1 Expresii regulate . . . 6
2.1.1 Utilizări . . . 7
2.2 Automate finite deterministe . . . 7
2.2.1 Utilizări . . . 8
2.3 Automate finite nedeterministe . . . 8
2.3.1 Utilizări . . . 10
2.4 Echivalența ER, AFD, AFN . . . 10
2.4.1 Conversie AFD $\rightarrow$ AFN . . . 10
2.4.2 Conversie AFN $\rightarrow$ AFD . . . 10
2.4.3 Conversie ER $\rightarrow$ AFN . . . 11
2.5 Proprietăți algoritmice . . . 12
2.5.1 Închidere în raport cu reuniunea . . . 12
2.5.2 Închidere în raport cu concatenarea . . . 12
2.5.3 Închidere în raport cu Kleene Star . . . 13
2.5.4 Închidere în raport cu diferența . . . 13
2.5.5 Închidere în raport cu intersecția . . . 14
2.5.6 Închidere în raport cu complementarea . . . 15
2.5.7 Decizia dacă un limbaj regulat este vid . . . 15
2.5.8 Decizia dacă un șir este într-un limbaj regulat . . . 15
2.5.9 Decizia dacă un limbaj regulat este finit . . . 16
2.6 Lema de pompare . . . 16
2.6.1 Enunț . . . 16
2.6.2 Negația Lemei de pompare . . . 17

2.6.3 Utilizare 17

3 Limbaje independente de context **18**
3.1 Gramatici independente de context 18
3.1.1 Utilizări 19
3.2 Automate Pushdown nedeterministe 19
3.2.1 Utilizări 20
3.3 Echivalența GIC, APD 20
3.3.1 Conversia GIC → APD 20
3.3.2 Conversia APD → GIC 20
3.4 Proprietăți algoritmice 20
3.4.1 Închiderea în raport cu reuniunea 20
3.4.2 Închiderea în raport cu concatenarea 21
3.4.3 Închiderea în raport cu Kleene Star 22
3.4.4 Clasa limbajelor independente de context nu este închisă în raport cu diferența, intersecția și complementarea . . 23
3.4.5 Decizia dacă un limbaj independent de context este vid 23
3.5 Lema de pompare 24
3.5.1 Enunț 24
3.5.2 Negația Lemei de pompare 24
3.5.3 Utilizare 24

1 Noțiuni generale

1.1 Alfabet, Șir, Limbaj

1.1.1 Alfabet

Un alfabet este orice mulțime nevidă finită de simboluri. Acestea pot fi caractere, semnale sau orice obiect ce poate fi interpretat cumva.
Exemple:

- alfabetul roman: $\Sigma = \{A, B, C, ..., Z\}$
- alfabetul format din cifre: $\Sigma = \{0, 1, 2, ..., 9\}$
- alfabetul semnalelor pentru 3-Way Handshake: $\Sigma = \{$SYN, ACK, SYN-ACK$\}$

1.1.2 Șir

Un șir(cuvânt) este o secvență finită de simboluri dintr-un alfabet. Un șir poate să nu conțină niciun caracter. Acesta, șirul vid, se notează cu e(în alte surse se notează cu ε).
Lungimea unui șir w se notează cu $|w|$ și reprezintă numărul de simboluri din care e format w.
Exemple:

- $\Sigma = \{a, b, c, d\}, w = cbba, |w| = 4$
- $\Sigma = \{0, 1\}, w = 10010101, |w| = 8$
- $\Sigma = \{a, b\}, w = e, |w| = 0$

1.1.3 Limbaj

Un limbaj peste un alfabet Σ este o mulțime de cuvinte formate din simboluri din Σ. Limbajul vid se notează cu $\emptyset$, și este limbajul în care nu se află niciun cuvânt. Acesta este diferit de limbajul care conține doar șirul vid. Un limbaj poate fi finit sau infinit.
Exemple:

- $L = \emptyset, |L| = 0$

- $L = \{e\}, |L| = 1$
- $L = \{e, a, ab\}, |L| = 3$
- $L = \{a, aa, aaa, ...\}, |L| = \infty$

1.2 Operații pe limbaje

1.2.1 Reuniune

Reuniunea a două limbaje este reuniunea normală definită pe mulțimi.

1.2.2 Concatenare

Fie L_1, L_2, două limbaje. Concatenarea se definește în felul următor:
$L_1 \circ L_2 = \{w_1w_2 | w_1 \in L_1 \wedge w_2 \in L_2\}$.
Mai exact, orice concatenare a unui cuvânt din L_1 cu un cuvânt din L_2 se află în $L_1 \circ L_2$. Concatenarea se poate nota mai simplu L_1L_2.
Concatenarea nu este comutativă.
Concatenarea este asociativă și distributivă la stânga, respectiv la dreapta față de reuniune.
Exemple:

- $L_1 = \emptyset$, $L_2 = \{e, a, b\}$. $L_1L_2 = \emptyset$, fiindcă nu există niciun cuvânt în L_1 care să fie concatenat cu vreun cuvânt din L_2
- $L_1 = \{a, b\}$, $L_2 = \{c, d\}$. $L_1L_2 = \{ac, ad, bc, bd\}$
- $L_1 = \{e, a\}$, $L_2 = \{b, bb, bbb, ...\}$. $L_1L_2 = \{b, ab, bb, abb, ...\}$

1.2.3 Kleene Star

Fie L un limbaj. Kleene Star se definește în felul următor:
$L^* = \{w | w = w_1w_2...w_n \wedge w_i \in L, n \geq 0\}$
Kleene Star aplicat pe un limbaj reprezintă concatenarea de zero sau mai multe ori a oricâtor șiruri din L.
Fie Σ un alfabet oarecare. Σ este o mulțime finită de simboluri. Acesta poate fi tratat ca un limbaj format din șiruri de lungime 1. Aplicând Kleene Star pe acest limbaj, obținem Σ^*, mulțimea tuturor șirurilor formate din simboluri din Σ.

Deoarece L^* conține concatenarea tuturor cuvintelor de zero sau mai multe ori, L^* îl va conține întotdeauna pe e.
Exemple:

- $L = \emptyset$. $L^* = \{e\}$.
- $L = \{e\}$. $L^* = \{e\}$.
- $L = \{a\}$. $L^* = \{e, a, aa, aaa, ..\}$
- $L = \{a, b\}$. $L^* = \{e, a, b, aa, ab, ba, bb, aaa, ...\}$

1.2.4 Mulțimea putere

Fie M o mulțime. Notăm cu 2^M mulțimea tuturor submulțimilor lui M.
$2^M = \{A | A \subseteq M\}$.
Exemplu: $M = \{0, 1\}$. $2^M = \{\emptyset, \{0\}, \{1\}, \{0, 1\}\}$
Dacă Σ este un alfabet, Σ^* reprezintă mulțimea tuturor șirurilor peste acel alfabet și 2^{Σ^*} este mulțimea tuturor limbajelor peste acel alfabet. Σ este finit. Σ^* este infinit numărabil. 2^{Σ^*} este infinit și nenumărabil.

1.3 Ierarhia Chomsky

Ierarhia constă în patru mari clase de limbaje:

1. Limbaje regulate
2. Limbaje independente de context
3. Limbaje dependente de context
4. Limbaje zero

2 Limbaje regulate

Clasa limbajelor regulate este mulțimea minimă de limbaje închisă în raport cu operațiile de reuniune, concatenare și Kleene Star.

2.1 Expresii regulate

Expresiile regulate reprezintă o metodă de reprezentare finită a limbajelor. Ele pot descrie orice succesiune finită de operații de reuniune, concatenare și Kleene Star.

Fie Σ un alfabet. O expresie regulată este o secvență finită de simboluri din $\Sigma \cup \{\cup, ^*, (,), \varnothing, e\}$, ținând cont de următoarele proprietăți:

1. $\varnothing$ și e sunt expresii regulate, reprezentând limbajul vid, $L_1 = \emptyset$, respectiv limbajul care conține doar șirul vid, $L_2 = \{e\}$;

2. $\forall a \in \Sigma$, a este expresie regulată, reprezentând limbajul ce conține un singur cuvânt format din simbolul a, $L = \{a\}$;

3. $\forall \alpha, \beta$ expresii regulate, $\alpha \cup \beta$ este expresie regulată, reprezentând reuniunea limbajelor descrise de expresiile regulate α și β, $\mathcal{L}(\alpha \cup \beta) = \mathcal{L}(\alpha) \cup \mathcal{L}(\beta)$;

4. $\forall \alpha, \beta$ expresii regulate, $\alpha\beta$ este expresie regulată, reprezentând concatenarea limbajelor descrise de expresiile regulate α și β, $\mathcal{L}(\alpha\beta) = \mathcal{L}(\alpha) \circ \mathcal{L}(\beta)$;

5. $\forall \alpha$ expresie regulată, α^* este expresie regulată, reprezentând aplicarea operatorului Kleene Star limbajului descris de expresia regulată α, $\mathcal{L}(\alpha^*) = \mathcal{L}(\alpha)^*$;

6. $\forall \alpha$ expresie regulată, (α) este expresie regulată. Parantezele cresc prioritatea operatorilor. Operatorii, ordonați de la prioritate maximă la minimă sunt: Kleene Star, operator de concatenare, operator de reuniune;

7. Orice altceva nu este expresie regulată.

Limbajele acceptate de expresii regulate se numesc limbaje regulate.
Notații ajutătoare:

- Fie α o expresie regulată. Notăm $\alpha^+ = \alpha\alpha^*$.
- Fie α o expresie regulată. Notăm $\alpha? = (\alpha \cup e)$

Exemple:

- $E = e \cup ab \cup bb^*$. $\mathcal{L}(E) = \mathcal{L}(e) \cup \mathcal{L}(ab) \cup \mathcal{L}(bb*) = \{e\} \cup \{ab\} \cup \mathcal{L}(b) \circ \mathcal{L}(b^*) = \{e, ab\} \cup \{b\} \circ \mathcal{L}(b)^* = \{e, ab\} \cup \{b\} \circ \{b\}^* = \{e, ab\} \cup \{b\} \circ \{e, b, bb, bbb, ...\} = \{e, ab\} \cup \{b, bb, bbb, bbbb, ...\} = \{e, ab, b, bb, bbb, ...\}$;
- $E = (a \cup b)^*$. $\mathcal{L}(E) = \mathcal{L}((a \cup b)^*) = \{a, b\}^* = \{e, a, b, aa, ab, ba, bb, aaa, ...\}$;
- $E = a^* \cup b^*$. $\mathcal{L}(E) = \mathcal{L}(a^*) \cup \mathcal{L}(b^*) = \{e, a, aa, aaa, ...\} \cup \{e, b, bb, bbb, ...\} = \{e, a, aa, aaa, ..., b, bb, bbb, ...\}$;

2.1.1 Utilizări

Expresiile regulate sunt folosite în motoare de căutare, unelte de prelucrare text (grep), limbaje de programare (haskell).

2.2 Automate finite deterministe

Un automat finit este un model de calcul care primește la intrare o bandă de simboluri și își modifică starea internă în funcție de ce are la intrare. Acesta poate fi privit ca o cutie neagră cu niște stări între care face tranziții în funcție de intrare, la fiecare tranziție mișcând capul de citire la dreapta.

Formal, un automat finit determinist este un tuplu $M = (K, \Sigma, \delta, s, F)$, cu următoarele proprietăți:

- K este mulțimea stărilor. K este finită, de unde vine și numele automatului;
- Σ este alfabetul din care sunt formate cuvintele acceptate de automat;
- δ se numește funcție de tranziție. $\delta : K \times \Sigma \to K$. $\delta(p, a) = q; p, q \in K; a \in \Sigma$ înseamnă că automatul, dacă se află în starea p și primește pe bandă a, trece în starea q. Fiindcă δ este funcție, toate tranzițiile din fiecare stare pentru fiecare simbol trebuie să fie definite.
- $s \in K$ este starea de start. Aceasta este starea în care se află automatul înainte de a primi ceva pe bandă.

- $F \subseteq K$ este mulțimea stărilor finale. Dacă automatul se află într-o stare $f \in F$ după ce a terminat de citit șirul înseamnă că acceptă șirul.

Un moment din execuția unui automat finit se numește configurație. O configurație completă constă într-un tuplu (q, w) format din starea curentă și șirul de la intrare. O configurație este un element din $K \times \Sigma^*$.

Pentru a se descrie trecerea între configurații se folosește operatorul $\vdash_M$: $K \times \Sigma^* \to K \times \Sigma^*$. $\forall q, q' \in K, \forall w, w' \in \Sigma^*, (q, w) \vdash_M (q', w') \Leftrightarrow \exists \sigma \in \Sigma, w = \sigma w' \wedge \delta(q, \sigma) = q'$

Se notează cu $\vdash_M^*$ închiderea reflexivă și tranzitivă a operatorului de trecere între configurații. Adică $\vdash_M^*$ reprezintă trecerea între configurații în zero sau mai mulți pași.

Un automat finit determinist acceptă un șir dacă și numai dacă la terminarea șirului de la intrare automatul se află într-o stare finală. $\forall w \in \Sigma^*, M$ acceptă $w \Leftrightarrow \exists f \in F, (s, w) \vdash_M^* (f, e)$.

Mulțimea tuturor cuvintelor acceptate de un automat se numește limbaj acceptat de automat. $\forall w \in \Sigma^*, w \in \mathcal{L}(M) \Leftrightarrow \exists f \in F, (s, w) \vdash_M^* (f, e)$. Un limbaj acceptat de un automat finit determinist este regulat.

2.2.1 Utilizări

Automatele finite deterministe sunt folosite la crearea analizoarelor lexicale, la proiectarea circuitelor electrice, la testarea protocoalelor, la crearea unor programe în care este evidentă existența unei stări interne.

2.3 Automate finite nedeterministe

Un automat finit nedeterminist este un automat finit mai puțin restrictiv decât unul determinist. Acesta se bazează pe faptul că se pot rula în paralel o infinitate de ramuri de execuție și dacă cel puțin una se termină într-o stare finală, șirul este acceptat.

Un automat finit nedeterminist permite:

- absența tranzițiilor pe anumite simboluri. Dacă se întâmplă ca la un moment dat să apară un simbol pentru care nu există tranziție, ramura curentă de execuție se termină fără a accepta șirul;

- existența mai multor tranziții din aceeași stare pe același caracter. Dacă se întâmplă acest lucru, automatul lansează mai multe ramuri de execuție, câte una pentru fiecare tranziție posibilă;

- tranziții pe șiruri. Fiindcă se pot neglija tranziții, un lanț de tranziții în care din stările intermediare nu pleacă nicio tranziție se poate considera o singură tranziție pe un șir.
- tranziții pe șirul vid. Automatul își poate modifica spontan starea fără a citi vreun caracter.

Un automat nedeterminist este tot un tuplu $M = (K, \Sigma, \Delta, s, F)$ cu următoarele proprietăți:

- K este mulțimea finită a stărilor;
- Σ este alfabetul din care sunt formate cuvintele acceptate de automat;
- $\Delta \subset (K \times \Sigma^* \times K)$ nu este funcție, este relație. Δ este o finită. $(p, u, q) \in \Delta$ înseamnă că automatul poate trece din starea p în starea q citind șirul u;
- $s \in K$ este starea de start;
- $F \subseteq K$ este mulțimea stărilor finale.

O configurație a unui automat finit nedeterminist se notează la fel ca în cazul determinist, $(q, w) \in (K \times \Sigma^*)$.

Operatorul de trecere între configurații se notează la fel ca în cazul determinist, $\vdash_M$, dar fiindcă se poate ajunge în mai multe configurații sau niciuna nu mai e funcție. Închiderea reflexivă și tranzitivă a operatorului de trecere între configurații se notează tot cu $\vdash_M^*$.

Un automat finit nedeterminist acceptă un șir dacă există cel puțin o cale astfel încât din configurația inițială să se ajungă într-o stare finală fără niciun caracter rămas la intrare. Limbajul acceptat de un automat finit nedeterminist este format din totalitatea șirurilor acceptate de el. $\forall w \in \Sigma^*, w \in \mathcal{L}(M) \Leftrightarrow \exists f \in F, (s, w) \vdash_M^* (f, e)$

Un automat determinist este un automat finit nedeterminist care are niște restricții impuse. Deși mulțimea automatelor finite deterministe este inclusă în mulțimea automatelor finite nedeterministe, limbajele acceptate de ele sunt aceleași, limbajele regulate.

2.3.1 Utilizări

Automatele finite nedeterministe pot fi folosite la modelarea unei probleme pentru a fi ulterior transformat în automat determinist, care e mai greu de scris direct.

2.4 Echivalența ER, AFD, AFN

2.4.1 Conversie AFD $\rightarrow$ AFN

Un automat finit determinist este și nedeterminist, deci conversia este trivială. Îl lăsăm exact asa cum e.

2.4.2 Conversie AFN $\rightarrow$ AFD

Fie $M_1 = (K_1, \Sigma_1, \Delta_1, s_1, F_1)$. Să se găsească un $M_2 = (K_2, \Sigma_2, \delta_2, s_2, F_2), \mathcal{L}(M_1) = \mathcal{L}(M_2), \Sigma_2 = \Sigma_1$.

Pași:

1. Eliminarea tranzițiilor pe șiruri.

   Fie $\Delta_1' = \{(p, c, q) | \forall (p, c, q) \in \Delta_1 \wedge |c| \leq 1\} \cup \{(p, c_1, q_{w1}), (q_{w1}, c_2, q_{w2}), ..., (q_{wn}, c_n, q) | \forall (p, w, q) \in \Delta_1 \wedge |w| = n > 1 \wedge w = c_1 c_2 ... c_n \wedge q_{w1}, q_{w2}, ..., q_{wn} \notin K_1\}$ și $K_1' = \{q_{w1}, q_{w2}, ...q_{w2} | (p, c_1, q_{w1}), (q_{w1}, c_2, q_{w2}), ..., (q_{wn}, c_n, q) \in \Delta_1' \setminus \Delta_1\}$.

   Fie $M_1' = (K_1', \Sigma_1, \Delta_1', s_1, F_1)$.

   $\mathcal{L}(M_1) = \mathcal{L}(M_1')$ fiindcă nu se schimbă nimic la tranzițiile normale, și rezultatul unui șir de tranziții pe câte un caracter e identic cu rezultatul tranziției pe șir.

2. Se construiește automatul determinist folosind ca stări închiderea reflexivă și tranzitivă operatorului de trecere între configurații restricționat la tranziții pe șirul vid. $K_2 = 2^{K_1}$.

   Fie $Q(q) = \{p | (q, w) \vdash_M^* (p, w)\}$. $Q(q)$ reprezintă totalitatea stărilor în care se poate ajunge pornind din q fără a citi vreun caracter.

   $s_2 = Q_0 = Q(s_1)$

   $\delta_2 : K_2 \times \Sigma \rightarrow K_2$. Adică $\delta_2 : 2^{K_1} \times \Sigma \rightarrow 2^{K_1}$. Pentru fiecare element $Q_i \in K_2$ se calculează $\delta(Q_i)$.

$F_2 = \{Q(f) | f \in F_1\}$.

Din cauza modului în care se face transformarea, numărul stărilor unui automat finit determinist tinde să fie crescut exponențial la trecerea de la AFN la AFD. $|K_2| = |2^{K_1}| = 2^{|K_1|}$.

2.4.3 Conversie ER → AFN

Pentru conversia aceasta vom trata expresiile regulate în același mod ca în cazul definirii; întâi constructori de bază și apoi operațiile de reuniune, concatenare și Kleene Star.

Toate automatele create acum vor avea o stare inițială și o singură stare finală. La combinarea a două automate stările inițiale și finale își pierd proprietatea de stare inițială, respectiv finală.

Pentru a fi mai ușor se definesc următoarea notație:

- $\forall E$ expresie regulată, $AFN(E)$ este automatul obținut prin aplicarea algoritmului de aici;

1. $E_1 = \varnothing$. $M_1 = (K_1, \Sigma_1, \Delta_1, s_1, F_1)$; $K_1 = \{s, f\}, \Delta_1 = \emptyset, F = \{f\}$;
   $E_2 = e$. $M_2 = (K_2, \Sigma_2, \Delta_2, s_2, F_2)$; $K_2 = \{s, f\}, \Delta_2 = \{(s, e, f)\}, F = \{f\}$;
2. $\forall a \in \Sigma, E = a$. $M = (K, \Sigma, \Delta, s, F)$; $K = \{s, f\}, \Delta = \{(s, a, f)\}, F = \{f\}$;
3. $\forall \alpha, \beta$ expresii regulate, $E = \alpha \cup \beta$. Fie $M_1 = AFN(\alpha) = (K_1, \Sigma_1, \Delta_1, s_1, \{f_1\})$, $M_2 = AFN(\beta) = (K_2, \Sigma_2, \Delta_2, s_2, \{f_2\})$. $M = (K, \Sigma, \Delta, s, F)$; $K = \{s, f\} \cup K_1 \cup K_2, \Delta = \Delta_1 \cup \Delta_2 \cup \{(s, e, s_1), (s, e, s_2), (f_1, e, f), (f_2, e, f)\}, F = \{f\}$;
4. $\forall \alpha, \beta$ expresii regulate, $E = \alpha\beta$. Fie $M_1 = AFN(\alpha) = (K_1, \Sigma_1, \Delta_1, s_1, \{f_1\})$, $M_2 = AFN(\beta) = (K_2, \Sigma_2, \Delta_2, s_2, \{f_2\})$. $M = (K, \Sigma, \Delta, s, F)$; $K = \{s, f\} \cup K_1 \cup K_2, \Delta = \Delta_1 \cup \Delta_2 \cup \{(s, e, s_1), (f_1, e, s_2), (f_2, e, f)\}, F = \{f\}$;
5. $\forall \alpha$ o expresie regulată, $E = \alpha^*$. Fie $M_1 = AFN(\alpha) = (K_1, \Sigma_1, \Delta_1, s_1, \{f_1\})$. $M = (K, \Sigma, \Delta, s, F)$; $K = \{s, f\} \cup K_1, \Delta = \Delta_1 \cup \{(s, e, f), (s, e, s_1), (f_1, e, f)\}$, $F = \{f\}$;

2.5 Proprietăți algoritmice

2.5.1 Închidere în raport cu reuniunea

Mulțimea limbajelor regulate este închisă în raport cu reuniunea. $\forall L_1, L_2$ limbaje regulate, $L_1 \cup L_2$ este regulat.

1. Expresii regulate:

   $\forall L_1, L_2, \exists E_1, E_2, \mathcal{L}(E_1) = L_1, \mathcal{L}(E_2) = L_2$.

   $\mathcal{L}(E_1 \cup E_2) = \mathcal{L}(E_1) \cup \mathcal{L}(E_2) = L_1 \cup L_2$.

   Se poate scrie o expresie regulată, $E = E_1 \cup E_2$, care descrie limbajul $L_1 \cup L_2$.

2. Automate finite nedeterministe:

   Fie $M_1 = (K_1, \Sigma_1, \Delta_1, s_1, F_1), M_2 = (K_2, \Sigma_2, \Delta_2, s_2, F_2)$, două automate finite, $\mathcal{L}(M_1) = L_1, \mathcal{L}(M_2) = L_2$. $K_1 \cap K_2 = \emptyset$, altfel, redenumim stările dintr-un automat.

   $M = (K, \Sigma, \Delta, s, F)$, $K = K_1 \cup K_2 \cup \{s\}, \Sigma = \Sigma_1 \cup \Sigma_2, \Delta = \Delta_1 \cup \Delta_2 \cup \{(s, e, s_1), (s, e, s_2)\}, F = F_1 \cup F_2$, acceptă $\mathcal{L}(L_1) \cup \mathcal{L}(L_2)$.

   Automatul ori trece în stările corespunzătoare automatului M_1 și acceptă un cuvânt din L_1, ori trece în stările corespunzătoare lui M_2 și acceptă un cuvânt din M_2.

2.5.2 Închidere în raport cu concatenarea

Mulțimea limbajelor regulate este închisă în raport cu concatenarea. $\forall L_1, L_2$ limbaje regulate, $L_1 \circ L_2$ este regulat.

1. Expresii regulate:

   $\forall L_1, L_2, \exists E_1, E_2, \mathcal{L}(E_1) = L_1, \mathcal{L}(E_2) = L_2$.

   $\mathcal{L}(E_1 E_2) = \mathcal{L}(E_1) \circ \mathcal{L}(E_2) = L_1 \circ L_2$.

   Se poate scrie o expresie regulată, $E = E_1 E_2$, care descrie limbajul $L_1 \circ L_2$.

2. Automate finite nedeterministe:

Fie $M_1 = (K_1, \Sigma_1, \Delta_1, s_1, F_1), M_2 = (K_2, \Sigma_2, \Delta_2, s_2, F_2)$, două automate finite, $\mathcal{L}(M_1) = L_1, \mathcal{L}(M_2) = L_2$. $K_1 \cap K_2 = \emptyset$, altfel, redenumim stările dintr-un automat.

$M = (K, \Sigma, \Delta, s, F)$, $K = K_1 \cup K_2, \Sigma = \Sigma_1 \cup \Sigma_2, \Delta = \Delta_1 \cup \Delta_2 \cup \{(f_1, e, s_2) | \forall f_1 \in F_1\}, s = s_1, F = F_2$, acceptă $\mathcal{L}(L_1) \circ \mathcal{L}(L_2)$.

Automatul trece prin stările corespunzătoare lui M_1 până ajunge într-o stare care era finală, apoi trece în s_2 și se oprește într-o stare finală după ce primește și un cuvânt din L_2.

2.5.3 Închidere în raport cu Kleene Star

Mulțimea limbajelor regulate este închisă în raport cu Kleene Star. $\forall L$ limbaj regulat, L'^* este regulat.

1. Expresii regulate:

   $\forall L', \exists E', \mathcal{L}(E') = L'$.

   $\mathcal{L}(E'^*) = \mathcal{L}(E')^* = L'^*$.

   Se poate scrie o expresie regulată, $E = E'^*$, care descrie limbajul L'^*.

2. Automate finite nedeterministe:

   Fie $M' = (K', \Sigma', \Delta', s', F')$, un automat finit, $\mathcal{L}(M') = L'$.

   $M = (K, \Sigma, \Delta, s, F)$, $K = K' \cup \{s\}, \Sigma = \Sigma', \Delta = \Delta' \cup \{(s, e, s')\} \cup \{(f', e, s) | \forall f' \in F'\}, F = \{s\}$, acceptă $\mathcal{L}(L')^*$.

   Automatul termină direct în s sau trece prin stările corespunzătoare lui M până ajunge într-o stare care era finală, de unde ajunge pe o tranziție vidă înapoi în s unde poate termina sau repeta procesul.

2.5.4 Închidere în raport cu diferența

Mulțimea limbajelor regulate este închisă în raport cu diferența. $\forall L_1, L_2$ limbaje regulate, $L_1 \setminus L_2$ este regulat.

1. Automate finite deterministe:

   Fie $M_1 = (K_1, \Sigma_1, \delta_1, s_1, F_1), M_2 = (K_2, \Sigma_2, \delta_2, s_2, F_2)$, două automate finite, $\mathcal{L}(M_1) = L_1, \mathcal{L}(M_2) = L_2$.

$\Sigma_1 = \Sigma_2 = \Sigma$, altfel unele tranziții necesare nu există. Dacă nu există, se pot completa Σ_1 și Σ_2 până la $\Sigma_1 \cup \Sigma_2$, introducând o stare de eroare nefinală în care să rămână pentru orice caracter și tranziții către acea stare din orice altă stare pentru orice caracter nou introdus.

$M = (K, \Sigma, \delta, s, F)$, $K = K_1 \times K_2, \delta = \{((q_1, q_2), c, (\delta(q_1), \delta(q_2)))|\forall q_1 \in K_1, \forall q_2 \in K_2, \forall c \in \Sigma\}, s = (s_1, s_2), F = \{(f_1, q_2)|\forall f_1 \in F_1, \forall q_2 \in K_2 \setminus F_2\}$, acceptă $\mathcal{L}(L_1) \setminus \mathcal{L}(L_2)$.

Automatul simulează execuția ambelor automate în paralel și acceptă un șir dacă doar primul automat ar fi ajuns într-o stare finală.

2.5.5 Închidere în raport cu intersecția

Mulțimea limbajelor regulate este închisă în raport cu intersecția. $\forall L_1, L_2$ limbaje regulate, $L_1 \cap L_2$ este regulat.

Demonstrație:

- Se poate considera $L_1 \cap L_2 = L_1 \setminus (L_1 \setminus L_2)$ și se folosește metoda de mai sus pentru automate.
- Se folosește următoarea metodă:

  1. Automate finite deterministe:

     Fie $M_1 = (K_1, \Sigma_1, \delta_1, s_1, F_1), M_2 = (K_2, \Sigma_2, \delta_2, s_2, F_2)$, două automate finite, $\mathcal{L}(M_1) = L_1, \mathcal{L}(M_2) = L_2$.

     $\Sigma_1 = \Sigma_2 = \Sigma$, altfel unele tranziții necesare nu există. Dacă nu există, se pot completa Σ_1 și Σ_2 până la $\Sigma_1 \cup \Sigma_2$, introducând o stare de eroare nefinală în care să rămână pentru orice caracter și tranziții către acea stare din orice altă stare pentru orice caracter nou introdus.

     $M = (K, \Sigma, \delta, s, F), K = K_1 \times K_2, \delta = \{((q_1, q_2), c, (\delta(q_1), \delta(q_2)))|\forall q_1 \in K_1, \forall q_2 \in K_2, \forall c \in \Sigma\}, s = (s_1, s_2), F = \{(f_1, f_2)|\forall f_1 \in F_1, \forall f_2 \in F_2\}$, acceptă $\mathcal{L}(L_1) \cap \mathcal{L}(L_2)$.

     Automatul simulează execuția ambelor automate în paralel și acceptă un șir când ambele automate ar fi ajuns într-o stare finală.

2.5.6 Închidere în raport cu complementarea

Mulțimea limbajelor regulate este închisă în raport cu complementarea. $\forall L'$ limbaj regulat, $L = L'^c = \bar{L'}$ este regulat.

Demonstrație:

- Se poate considera $L = \Sigma^* \setminus L'$ și se aplică metoda menționată anterior pentru cele două automate.
- Se folosește următoarea metodă:

  1. Automate finite deterministe:

     Fie $M' = (K', \Sigma', \delta', s', F')$, un automat finit determinist, $\mathcal{L}(M') = L'$.

     $M = (K', \Sigma', \delta', s', K' \setminus F')$ acceptă L'^c.

     Complementând mulțimea stărilor finale se obține exact complementul limbajului.

2.5.7 Decizia dacă un limbaj regulat este vid

1. Expresii regulate:

   Fie E o expresie regulată $\mathcal{L}(E) = \emptyset \Leftrightarrow \exists \alpha, \beta \in \Sigma^*, E = \alpha \varnothing \beta$. Dacă $\alpha = \beta = e$, atunci $E = \varnothing$.

2. Automate finite deterministe sau nondeterministe:

   Pentru a stabili dacă automatul acceptă vreun șir, se face o parcurgere a grafului și dacă se ajunge într-o stare finală limbajul nu e vid.

2.5.8 Decizia dacă un șir este într-un limbaj regulat

1. Expresii regulate:

   Se generează în ordinea lungimii șirurile acceptate de expresie și se compară cu șirul căutat.

2. Automate finite deterministe:

   Se lansează în execuție automatul cu șirul la intrare și se așteaptă sfârșitul execuției automatului. Dacă la final este într-o stare finală șirul este acceptat.

3. Automate finite nedeterministe:

   Se lansează în execuție automatul cu șirul la intrare. Dacă vreuna din ramuri se termină într-o stare finală șirul este acceptat. Pentru a se asigura terminarea execuției, ciclurile care ajung în starea de unde au pornit fără a fi citit vreun caracter se execută o singură dată.

2.5.9 Decizia dacă un limbaj regulat este finit

1. Expresii regulate:

   Limbajul descris de o expresie regulată este finit dacă orice operație de Kleene Star din interior este într-o subexpresie care descrie limbajul vid. Limbajul descris de o expresie fără Kleene Star este finit. Limbajul vid este finit.

2. Automate finite deterministe sau nedeterministe:

   Un automat finit care conține o stare finală aflată în/după un ciclu accesibil din starea inițială acceptă o infinitate de șiruri.

2.6 Lema de pompare

Lema de pompare este o proprietate pe care o are orice limbaj regulat.

2.6.1 Enunț

Fie L un limbaj regulat. Atunci $\exists n_0 \in \mathbb{N}$ astfel încât $\forall w \in L, |w| \geq n_0$
$\exists x, y, z \in \Sigma^*, w = xyz, |xy| \leq n_0, |y| > 0$ astfel încât $\forall k \in \mathbb{N}, xy^k z \in L$.

Lema de pompare se poate formula astfel: Pentru orice limbaj regulat, orice șir destul de lung am avea, el se poate împărți în trei părți astfel încât pompând partea din centru de oricâte ori, șirul rezultat să rămână în limbaj.

Pentru limbaje finite se poate alege n_0 lungimea maximă din limbaj. Astfel, pentru toate șirurile mai lungi decât n_0, adică niciunul, se aplică lema.

Pentru o expresie regulată, pentru orice șir se alege un subșir care s-a potrivit o dată pentru o subexpresie căreia i se aplică Kleene Star și poate pompa de oricâte ori, șirul rezultat încă potrivindu-se expresiei regulate.

2.6.2 Negația Lemei de pompare

Lema de pompare nu poate fi folosită pentru a demonstra că un limbaj este regulat fiindcă există limbaje care nu sunt regulate pentru care lema de pompare funcționează.

Fiindcă lema funcționează pentru orice limbaj regulat, limbajele pentru care lema nu se aplică cu siguranță nu sunt regulate. Astfel, dintr-o proprietate fără mare utilitate, lema de pompare a devenit un instrument foarte bun de demonstrare a faptului că un limbaj nu ar fi regulat.

Forma negată a lemei de pompare este:

Fie L un limbaj. Dacă pentru $\forall n_0 \in \mathbb{N}$, $\exists w \in L, |w| \geq n_0$ astfel încât $\forall x, y, z \in \Sigma^*, w = xyz, |xy| \leq n_0, |y| > 0$, $\exists k \in \mathbb{N}$ astfel încât $xy^kz \notin L$, atunci L nu este regulat.

Pentru a demonstra că o proprietate nu e adevărată, căutăm un contraexemplu. Pentru orice dimensiune, căutăm un șir mai lung astfel încât oricum l-am împărți, pomparea șirului din mijloc produce un șir care nu e în limbaj.

2.6.3 Utilizare

Demonstrați folosind lema de pompare că $L = \{a^n b^m | n \neq m\}$ nu este regulat.

Rezolvare: Presupunem L regulat. Știind că $L' = \mathcal{L}(a^*b^*)$ este regulat, $L' \setminus L = \{a^n b^m | n = m\}$ ar trebui să fie regulat, deci ar trebui să respecte lema de pompare.

Fie n_0 oarecare, $w = a^{n_0}b^{n_0}, |w| = 2n_0 \geq n_0$.
$\forall x, y, z \in \Sigma^*, w = xyz$, din cauza restricțiilor $|xy| \leq n_0$ și $|y| > 0$, nu putem pompa decât a. Atunci, considerăm $x = a^i; y = a^j, j > 0; z = a^{n_0-i-j}b^{n_0}$. Atunci, $xy^kz = a^i a^{jk} a^{n_0-i-j} b^{n_0} = a^{n_0+j(k-1)}b^{n_0}$. Pentru $k = 2$, $xy^2z = a^{n_0+j}b^{n_0}$. Pentru $j \neq 0$, $n_0 + j \neq n_0$, adică $a^{n_0+j}b^{n_0} \notin L'$. Contradicție! $\Rightarrow$ presupunerea că L ar fi regulat e greșită.

3 Limbaje independente de context

Din păcate, nu toate limbajele sunt regulate. Pentru că expresiile regulate și automatele finite pot descrie doar limbaje regulate, a fost nevoie de ceva mai mult. Clasa limbajelor independente de context este următoarea clasă principală în ierarhia Chomsky.

3.1 Gramatici independente de context

Gramaticile independente de context reprezintă o altă metodă de reprezentare finită a limbajelor.

Limba română are o gramatică a ei. O gramatică reprezintă o mulțime de reguli de modificare și combinare a cuvintelor în propoziții corecte. Considerând noțiuni definite până acum, limba română poate fi asociată unui limbaj. Propozițiile corecte pot fi considerate șiruri din limbaj și componentele propozițiilor, cuvintele, pot fi considerate litere din alfabet. Astfel, facem următoarele asocieri:

- limbă → limbaj;
- propoziție → șir;
- cuvânt → simbol;
- vocabular → alfabet.

Analizate astfel, noțiunile definite până acum nu sunt suficiente pentru a defini o gramatică fiindcă nu se pot reprezenta regulile.

O gramatică este, din punct de vedere formal, un tuplu $G = (V, \Sigma, R, S)$, unde V reprezintă mulțimea simbolurilor ce pot apărea în reguli, Σ reprezintă alfabetul, R reprezintă mulțimea regulilor și S reprezintă simbolul de start.

Σ este o mulțime finită de simboluri care pot apărea în cuvintele limbajului, simboluri numite și terminali. V este mulțimea tuturor simbolurilor care apar în reguli, dar, pe lângă terminali, conține și niște simboluri care nu apar în cuvintele limbajului, numite neterminali. Un astfel de simbol este $S \in V \setminus \Sigma$.

R este mulțimea finită a regulilor de producție. O regulă este un tuplu, $(A, \gamma), A \in V \setminus \Sigma, \gamma \in V^*$, care înseamnă că neterminalul A poate fi înlocuit cu șirul de terminali și neterminali γ. Regula (A, γ) se notează cu $A \to \gamma$. Modul în care este definit face ca $R \subset (V \setminus \Sigma) \times V^*$. În cazul în care

$\exists A \in V \setminus \Sigma, \exists \gamma, \beta \in V^*, (A, \gamma) \in R \wedge (A, \beta) \in R$, atunci regulile se pot scrie $A \to \gamma|\beta$, însemnând că A se poate înlocui fie cu γ, fie cu β.

Înlocuirea părții din stânga a unei reguli cu partea din dreapta într-un șir se numește derivare. Spunem că v este derivat din $u (u \Rightarrow v)$ dacă $\exists A \in V \setminus \Sigma, \exists \alpha, \beta, \delta \in V^*$ astfel încât $(A, \delta) \in R \wedge u = \alpha A \beta \wedge v = \alpha \delta \beta$.

Închiderea reflexivă și tranzitivă a relației de derivare se notează cu $\Rightarrow^*$. $u \Rightarrow^* v$ înseamnă că v este derivat din u în zero sau mai mulți pași.

O gramatică generează un șir w dacă și numai dacă pornind de la simbolul de start se poate deriva w. Mulțimea tuturor șirurilor generate de o gramatică G se numește limbaj generat de gramatică. $w \in \mathcal{L}(G) \Leftrightarrow S \Rightarrow^* w$.

Un limbaj generat de o gramatică independentă de context se numește limbaj independent de context.

3.1.1 Utilizări

Gramaticile independente de context sunt folosite pentru a descrie formal limbaje de programare. Din cauza asta sunt foarte folosite în compilatoare.

3.2 Automate Pushdown nedeterministe

Din cauza faptului că automatele Pushdown deterministe pot accepta doar o submulțime a limbajelor independente de context nu vor fi folosite deloc.

Un automat Pushdown nedeterminist este un automat finit nedeterminist care are atașată o stivă.

Formal, un APD este un tuplu $M = (K, \Sigma, \Gamma, \Delta, s, F)$, unde K este mulțimea stărilor, Σ este alfabetul, Γ este alfabetul stivei, $\Delta \subset (K \times \Sigma^* \times \Gamma^*) \times (K \times \Gamma^*)$, este relația de tranziție între stări, s este starea de start și F este mulțimea stărilor finale.

Dacă $((p, u, \alpha), (q, \beta)) \in \Delta$, înseamnă că automatul poate trece din starea p în starea q, citind u de la intrare și scoțând α de pe stivă, punând în loc β.

O configurație a unui APD este un element $(p, w, \alpha) \in K \times \Sigma^* \times \Gamma^*$.

Operatorul de trecere între configurații este $\vdash_M \in K \times \Sigma^* \times \Gamma^* \times K \times \Sigma^* \times \Gamma^*$. Închiderea reflexivă și tranzitivă a operatorului de trecere între configurații este $\vdash_M^*$.

Un automat Pushdown nedeterminist acceptă un șir dacă există cel puțin o cale astfel încât din configurația inițială să se ajungă într-o stare finală fără niciun caracter rămas la intrare și cu stiva goală. Limbajul acceptat

de un automat Pushdown nedeterminist este format din totalitatea șirurilor acceptate de el. $\forall w \in \Sigma^*, w \in \mathcal{L}(M) \Leftrightarrow \exists f \in F, (s, w, e) \vdash_M^* (f, e, e)$.

3.2.1 Utilizări

Un APD poate fi folosit pentru implementarea unui parser după o gramatică dată.

3.3 Echivalența GIC, APD

3.3.1 Conversia GIC $\rightarrow$ APD

Fie $G = (V, \Sigma, R, S)$ o gramatică independentă de context. Să se găsească un automat Pushdown nedeterminist $M = (K, \Sigma, \Gamma, \Delta, s, F)$ care să accepte limbajul generat de G, $\mathcal{L}(M) = \mathcal{L}(G)$.

Vom scrie un automat ale cărui tranziții simulează derivările din gramatică.

$K = \{s, f\}$; $\Gamma = V$; $\Delta = \{((s, e, e), (f, S))\} \cup \{((f, e, A), (f, \gamma))|\forall A \in V, \gamma \in \Sigma^* \wedge (A, \gamma) \in R\} \cup \{((f, a, a), (f, e))|\forall a \in \Sigma\}$; $F = \{f\}$.

Automatul pune inițial pe stivă simbolul de start și trece în starea în care face derivările. O derivare este simulată scoțând de pe stivă neterminalul din stânga regulii și punerea pe stivă a părții din dreapta regulii. În cazul în care nu poate face o derivare fiindcă simbolul din vârful stivei este un terminal, automatul poate citi la intrare terminalul respectiv și îl scoate de pe stivă.

3.3.2 Conversia APD $\rightarrow$ GIC

3.4 Proprietăți algoritmice

3.4.1 Închiderea în raport cu reuniunea

Mulțimea limbajelor independente de context este închisă în raport cu reuniunea. $\forall L_1, L_2$ limbaje independente de context, $L_1 \cup L_2$ este independent de context.

1. Gramatici independente de context:

   Fie $G_1 = (V_1, \Sigma_1, R_1, S_1), G_2 = (V_2, \Sigma_2, R_2, S_2), \mathcal{L}(G_1) = L_1, \mathcal{L}(G_2) = L_2$. $(V_1 \setminus \Sigma_1) \cap (V_2 \setminus \Sigma_2) = \emptyset$, altfel redenumim neterminalii identici.

$G = (V, \Sigma, R, S)$. $V = V_1 \cup V_2$; $\Sigma = \Sigma_1 \cup \Sigma_2$; $R = R_1 \cup R_2 \cup \{(S, S_1), (S, S_2)\}$.

Din simbolul de start se poate deriva simbolul de start pentru una din cele două gramatici de unde se poate deriva orice cuvânt.

$\mathcal{L}(G) = \mathcal{L}(G_1) \cup \mathcal{L}(G_2)$.

2. Automate Pushdown nedeterministe:

   Fie $M_1 = (K_1, \Sigma_1, \Gamma_1, \Delta_1, s_1, F_1), M_2 = (K_2, \Sigma_2, \Gamma_2, \Delta_2, s_2, F_2)$, două automate Pushdown nedeterministe, $\mathcal{L}(M_1) = L_1, \mathcal{L}(M_2) = L_2$. $K_1 \cap K_2 = \emptyset$, altfel, redenumim stările dintr-un automat.

   $M = (K, \Sigma, \Gamma, \Delta, s, F)$, $K = K_1 \cup K_2 \cup \{s\}, \Sigma = \Sigma_1 \cup \Sigma_2, \Gamma = \Gamma_1 \cup \Gamma_2, \Delta = \Delta_1 \cup \Delta_2 \cup \{((s, e, e), (s_1, e)), ((s, e, e), (s_2, e))\}, F = F_1 \cup F_2$, acceptă $\mathcal{L}(L_1) \cup \mathcal{L}(L_2)$.

   Automatul ori trece în stările corespunzătoare automatului M_1 și acceptă un cuvânt din L_1, ori trece în stările corespunzătoare lui M_2 și acceptă un cuvânt din M_2.

3.4.2 Închiderea în raport cu concatenarea

Mulțimea limbajelor independente de context este închisă în raport cu concatenarea. $\forall L_1, L_2$ limbaje independente de context, $L_1 \circ L_2$ este independent de context.

1. Gramatici independente de context:

   Fie $G_1 = (V_1, \Sigma_1, R_1, S_1), G_2 = (V_2, \Sigma_2, R_2, S_2), \mathcal{L}(G_1) = L_1, \mathcal{L}(G_2) = L_2$. $(V_1 \setminus \Sigma_1) \cap (V_2 \setminus \Sigma_2) = \emptyset$, altfel redenumim neterminalii identici.

   $G = (V, \Sigma, R, S)$. $V = V_1 \cup V_2$; $\Sigma = \Sigma_1 \cup \Sigma_2$; $R = R_1 \cup R_2 \cup \{(S, S_1 S_2)\}$.

   Din simbolul de start se derivează concatenarea celor două simboluri de start, din fiecare derivându-se câte un cuvânt din limbajul respectiv.

   $\mathcal{L}(G) = \mathcal{L}(G_1) \circ \mathcal{L}(G_2)$.

2. Automate Pushdown nedeterministe:

   Fie $M_1 = (K_1, \Sigma_1, \Gamma_1, \Delta_1, s_1, F_1), M_2 = (K_2, \Sigma_2, \Gamma_2, \Delta_2, s_2, F_2)$, două automate Pushdown nedeterministe, $\mathcal{L}(M_1) = L_1, \mathcal{L}(M_2) = L_2$. $K_1 \cap K_2 = \emptyset$, altfel, redenumim stările dintr-un automat. $\Gamma_1 \cap \Gamma_2 = \emptyset$, altfel

redenumim simbolurile dintr-unul dintre alfabetele stivei unuia dintre automate.

$M = (K, \Sigma, \Gamma, \Delta, s, F)$, $K = K_1 \cup K_2, \Sigma = \Sigma_1 \cup \Sigma_2, \Gamma = \Gamma_1 \cup \Gamma_2, \Delta = \Delta_1 \cup \Delta_2 \cup \{((f_1, e, e), (s_2, e)) | \forall f_1 \in F_1\}, s = s_1, F = F_2$, acceptă $\mathcal{L}(L_1) \circ \mathcal{L}(L_2)$.

Automatul trece prin stările corespunzătoare lui M_1 până ajunge într-o stare care era finală, apoi trece în s_2 și se oprește într-o stare finală după ce primește și un cuvânt din L_2. Fiindcă simbolurile de pe stivă sunt diferite, al doilea automat nu poate curăța stiva dacă primul a lăsat ceva, așa că un șir e acceptat doar dacă amândouă automatele au lăsat stiva goală.

3.4.3 Închiderea în raport cu Kleene Star

Mulțimea limbajelor independente de context este închisă în raport cu Kleene Star. $\forall L'$ limbaj independent de context, $L = L'^*$ este independent de context.

1. Gramatici independente de context:

   Fie $G' = (V', \Sigma', R', S'), \mathcal{L}(G') = L'$.

   $G = (V, \Sigma, R, S)$. $V = V'$; $\Sigma = \Sigma'$; $R = R' \cup \{(S, S'S), (S, e)\}$.

   Din simbolul de start se derivează $S'S$, făcând practic o inserare a unui cuvânt din L' la începutul șirului. După ce se inserează numărul de cuvinte necesare se poate face o derivare a șirului vid din ultimul simbol de start.

   $\mathcal{L}(G) = \mathcal{L}(G_1)^*$.

2. Automate Pushdown nedeterministe:

   Fie $M' = (K', \Sigma', \Gamma', \Delta', s', F')$, un automat Pushdown nedeterminist, $\mathcal{L}(M') = L'$.

   $M = (K, \Sigma, \Gamma, \Delta, s, F)$, $K = K' \cup \{s, f\}, \Sigma = \Sigma', \Gamma = \Gamma' \cup \{Z\}, Z \notin \Gamma', \Delta = \Delta' \cup \{((s, e, e), (s', Z))\} \cup \{((f', e, Z), (s, e)) | \forall f' \in F'\}, F = F_2$, acceptă $\mathcal{L}(L_1)^*$.

   Automatul poate termina direct din starea s sau poate trece prin stările lui M'. Dacă trece prin stările lui M', pune întâi un simbol pe stivă pe

care M' nu îl poate scoate și nu îl scoate decât când dintr-o stare finală se întroarce în s, dar doar dacă automatul a ajuns în acea stare cu stiva curățată. Astfel, fiecare rulare a automatului acceptă un cuvânt.

3.4.4 Clasa limbajelor independente de context nu este închisă în raport cu diferența, intersecția și complementarea

Fie $L_1 = \{a^m b^n c^p | \forall m, n, p \in \mathbb{N} \wedge m = n\}, L_2 = \{a^m b^n c^p | \forall m, n, p \in \mathbb{N} \wedge n \neq p\}$. $L_1 \setminus L_2 = \{a^m b^n c^p | \forall m, n, p \in \mathbb{N} \wedge m = n = p\}$, care nu este independent de context.

Intersecția și complementarea se pot descrie în funcție de diferență astfel:

- $A \cap B = A \setminus (A \setminus B)$
- $A^c = \Sigma^* \setminus A$

3.4.5 Decizia dacă un limbaj independent de context este vid

1. Gramatici independente de context:

   Pentru a verifica dacă o gramatică independentă de context generează cel puțin un șir se marchează pe rând simbolurile din care se pot deriva șiruri de terminali.

   Algoritm:

   (a) Se marchează terminalii ca fiind simboluri din care se pot deriva șiruri formate doar din terminali.

   (b) La fiecare iterație se parcurg regulile și se marchează neterminalii din stânga regulilor care au în dreapta șiruri formate doar din simboluri deja marcate.

   (c) Dacă după o iterație nu s-a mai marcat niciun neterminal, algoritmul se oprește

   Dacă la finalul algoritmului simbolul de start, S, este marcat, înseamnă că gramatica poate genera cel puțin un șir fără neterminali, adică limbajul este nevid

3.5 Lema de pompare

Lema de pompare este o proprietate pe care o respectă orice limbaj independent de context.

3.5.1 Enunț

Fie L un limbaj independent de context. Atunci $\exists n_0 \in \mathbb{N}$ astfel încât $\forall w \in L, |w| \geq n_0$, $\exists u, v, x, y, z \in \Sigma^*, w = uvxyz, |vxy| \leq n_0, |vy| > 0$ astfel încât $\forall k \in \mathbb{N}, uv^kxy^kz \in L$.

Lema de pompare se poate formula astfel: Pentru orice limbaj independent de context, orice șir destul de lung am avea, el se poate împărți în cinci subșiruri astfel încât pompând simultan al doilea și al patrulea subșir de oricâte ori, șirul rezultat să rămână în limbaj.

3.5.2 Negația Lemei de pompare

Lema de pompare nu poate fi folosită pentru a demonstra că un limbaj este independent de context fiindcă există limbaje care nu sunt independente de cotnext pentru care lema de pompare funcționează.

Fiindcă lema funcționează pentru orice limbaj independent de context, limbajele pentru care lema nu se aplică, cu siguranță nu sunt independente de context. Astfel, dintr-o proprietate fără mare utilitate, lema de pompare a devenit un instrument foarte bun de demonstrare a faptului că un limbaj nu ar fi independent de context.

Forma negată a lemei de pompare este:

Fie L un limbaj. Dacă pentru $\forall n_0 \in \mathbb{N}$, $\exists w \in L, |w| \geq n_0$ astfel încât $\forall u, v, x, y, z \in \Sigma^*, w = uvxyz, |vxy| \leq n_0, |vy| > 0$, $\exists k \in \mathbb{N}$ astfel încât $uv^kxy^kz \notin L$, atunci L nu este independent de context.

Pentru a demonstra că o proprietate nu e adevărată, căutăm un contraexemplu. Pentru orice dimensiune, căutăm un șir mai lung astfel încât oricum l-am împărți, pomparea celor două subșiruri produce un șir care nu e în limbaj.

3.5.3 Utilizare

Demonstrați folosind lema de pompare că $L = \{a^nb^nc^n\}$ nu este independent de context.

Rezolvare:

Fie n_0 oarecare, $w = a^{n_0}b^{n_0}c^{n_0}, |w| = 3n_0 \geq n_0$.
$\forall u, v, x, y, z \in \Sigma^*, w = uvxyz$, din cauza restricțiilor $|vxy| \leq n_0$ și $|vy| > 0$, vxy poate să conțină doar a, doar b, doar c, doar a și b sau doar b și c.

Cazuri:

1. $u = a^i$

   $vxy = a^j; v = a^{j_v}, x = a^{j_x}, y = a^{j_y}; \ j_v + j_x + j_y = j \leq n, j_v + j_y > 0$

   $z = a^{n_0-i-j}b^{n_0}c^{n_0}$

   $uv^kxy^kz = a^ia^{kj_v}a^{j_x}a^{kj_y}a^{n_0-i-j}b^{n_0}c^{n_0} = a^{n_0+(j_v+j_y)(k-1)}b^{n_0}c^{n_0}$. Pentru $k = 2$, $n_0 + (j_v + j_y) \neq n_0$, adică $a^{n_0+(j_v+j_y)}b^{n_0}c^{n_0} \notin L$

2. $u = a^{n_0-i}$

   $vxy = a^ib^j; \ i + j \leq n_0; \ i > 0, j > 0$

   $z = b^{n_0-j}c^{n_0}$

   Cazuri:

   (a) $v = a^{i_v}; \ x = a^{i_x}b^{j_x}; \ y = b^{j_y}; \ i_v + i_x = i, j_x + j_y = j, i_v + j_y > 0$
   $uv^kxy^kz = a^{n_0-i}a^{ki_v}a^{i_x}b^{j_x}b^{kj_y}b^{n_0-j}c^{n_0} = a^{n_0+i_v(k-1)}b^{n_0+j_y(k-1)}c^{n_0}$.
   Pentru $k = 2$, $n_0+i_v \neq n_0 \vee n_0+j_y \neq n_0$, adică $a^{n_0+i_v}b^{n_0+j_y}c^{n_0} \notin L$

   (b) $v = a^ib^{j_v}; \ x = b^{j_x}; \ y = b^{j_y}; \ j_v + j_x + j_y = j, i + j_v + j_y > 0, j_v > 0$
   $uv^kxy^kz = a^{n_0-i}(a^ib^{j_v})^kb^{j_x}b^{kj_y}b^{n_0-j}c^{n_0}$. Pentru $k = 2$, se strică forma șirului, $a^{n_0}b^{j_v}a^ib^{n_0+j_y(k-1)}c^{n_0} \notin L$

   (c) $v = a^{i_v}; \ x = a^{i_x}; \ y = a^{i_y}b^j; \ i_v + i_x + i_y = i, i_v + i_y + j > 0, i_y > 0$
   $uv^kxy^kz = a^{n_0-i}a^{ki_v}a^{i_x}(a^{i_y}b^j)^kb^{n_0-j}c^{n_0}$. Pentru $k = 2$, se strică forma șirului, $a^{n_0+i_v(k-1)}b^ja^{i_y}b^{n_0}c^{n_0} \notin L$

3. $u = a^{n_0}b^i$

   $vxy = b^j; \ v = b^{j_v}, x = b^{j_x}, y = b^{j_y}; \ j_v + j_x + j_y = j \leq n, j_v + j_y > 0$

   $z = c^{n_0-i-j}$

   $uv^kxy^kz = a^{n_0}b^ib^{kj_v}b^{j_x}b^{kj_y}b^{n_0-i-j}c^{n_0} = a^{n_0}b^{n_0+(j_v+j_y)(k-1)}c^{n_0}$. Pentru $k = 2$, $n_0 + (j_v + j_y) \neq n_0$, adică $a^{n_0}b^{n_0+(j_v+j_y)}c^{n_0} \notin L$

4. $u = a^{n_0}b^{n_0-i}$

   $vxy = b^ic^j; \ i + j \leq n_0; \ i > 0, j > 0$

   $z = c^{n_0-j}$

   Cazuri:

(a) $v = b^{i_v}$; $x = b^{i_x}c^{j_x}$; $y = c^{j_y}$; $i_v + i_x = i, j_x + j_y = j, i_v + j_y > 0$
$uv^kxy^kz = a^{n_0}b^{n_0-i}b^{ki_v}b^{i_x}c^{j_x}c^{kj_y}c^{n_0-j} = a^{n_0}b^{n_0+i_v(k-1)}c^{n_0+j_y(k-1)}$.
Pentru $k = 2$, $n_0+i_v \neq n_0 \vee n_0+j_y \neq n_0$, adică $a^{n_0}b^{n_0+i_v}c^{n_0+j_y} \notin L$

(b) $v = b^ic^{j_v}$; $x = c^{j_x}$; $y = c^{j_y}$; $j_v + j_x + j_y = j, i + j_v + j_y > 0, j_v > 0$
$uv^kxy^kz = a^{n_0}b^{n_0-i}(b^ib^{j_v})^kc^{j_x}b^{kj_y}c^{n_0-j}$. Pentru $k = 2$, se strică forma șirului, $a^{n_0}b^{n_0}c^{j_v}b^ic^{n_0+j_y(k-1)} \notin L$

(c) $v = b^{i_v}$; $x = b^{i_x}$; $y = b^{i_y}c^j$; $i_v + i_x + i_y = i, i_v + i_y + j > 0, i_y > 0$
$uv^kxy^kz = a^{n_0}b^{n_0-i}b^{ki_v}b^{i_x}(b^{i_y}c^j)^kc^{n_0-j}$. Pentru $k = 2$, se strică forma șirului, $a^{n_0}b^{n_0+i_v(k-1)}c^jb^{i_y}c^{n_0} \notin L$

5. $u = a^{n_0}b^i$

$vxy = c^j$; $v = c^{j_v}, x = c^{j_x}, y = c^{j_y}$; $j_v + j_x + j_y = j \leq n, j_v + j_y > 0$

$z = c^{n_0-i-j}$

$uv^kxy^kz = a^{n_0}b^{n_0}c^ic^{kj_v}c^{j_x}c^{kj_y}c^{n_0-i-j} = a^{n_0}b^{n_0}c^{n_0+(j_v+j_y)(k-1)}$. Pentru $k = 2$, $n_0 + (j_v + j_y) \neq n_0$, adică $a^{n_0}b^{n_0}c^{n_0+(j_v+j_y)} \notin L$.